QCar2 Internship

Quang Thinh BUI

April 2026

Flatness-Based Bézier Trajectory Generation for QCar2

1 Introduction

For trajectory planning, the QCar2 is approximated by a car-like kinematic model, which provides a simpler flatness-based representation than the bicycle dynamic model. Let x and y denote the Cartesian coordinates of the midpoint of the rear axle in the inertial frame, θ the vehicle heading angle, and φ the front-wheel steering angle. Let v be the longitudinal velocity, ω^s the steering angular velocity, and l the wheelbase of the vehicle. Moreover, the Cartesian coordinates of the midpoint of the front axle are given by

$$x_f = x + l\cos\theta, \qquad y_f = y + l\sin\theta. \tag{1}$$

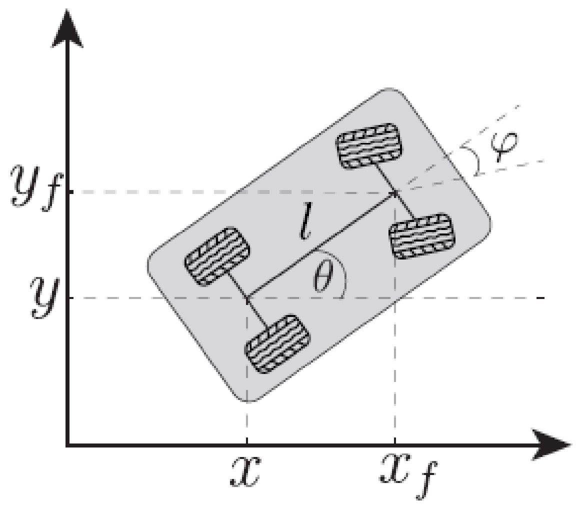

Figure 1: Car-like vehicle model.

The state and input vectors are defined as

$$\mathbf{x}_k = \begin{bmatrix} x & y & \theta & \varphi \end{bmatrix}^\top, \qquad \mathbf{u}_k = \begin{bmatrix} v & \omega^s \end{bmatrix}^\top.$$

Under the pure-rolling and no-slip assumptions, the car-like model is given by

$$\dot{x} = v\cos\theta \tag{2a}$$
$$\dot{y} = v\sin\theta \tag{2b}$$
$$\dot{\theta} = \frac{v}{l}\tan\varphi \tag{2c}$$
$$\dot{\varphi} = \omega^s. \tag{2d}$$

2 Differential Flatness

2.1 State and Input Representation in Terms of the Flat Output

For the car-like kinematic model, the flat output is chosen as the Cartesian position of the midpoint of the rear axle:

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix}. \tag{3}$$

Since

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta,$$

The first derivative of the flat output is

$$\dot{\mathbf{z}} = \begin{bmatrix} \dot{z}_1 \\ \dot{z}_2 \end{bmatrix} = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \end{bmatrix}.$$

Therefore, the longitudinal velocity and the vehicle heading angle can be directly recovered from the flat output as

$$v = \sqrt{\dot{z}_1^2 + \dot{z}_2^2}, \qquad \theta = \arctan\left(\frac{\dot{z}_2}{\dot{z}_1}\right). \tag{4}$$

To recover the steering angle, we differentiate the heading angle. From

$$\theta = \arctan\left(\frac{\dot{z}_2}{\dot{z}_1}\right),$$

Its time derivative is

$$\dot{\theta} = \frac{\dot{z}_1\ddot{z}_2 - \dot{z}_2\ddot{z}_1}{\dot{z}_1^2 + \dot{z}_2^2}.$$

From the car-like model,

$$\dot{\theta} = \frac{v}{l}\tan\varphi.$$

we obtain

$$\tan\varphi = \frac{l\left(\dot{z}_1\ddot{z}_2 - \dot{z}_2\ddot{z}_1\right)}{\left(\dot{z}_1^2 + \dot{z}_2^2\right)^{3/2}}. \tag{5}$$

Thus, the steering angle is given by

$$\varphi = \arctan\left(\frac{l\left(\dot{z}_1\ddot{z}_2 - \dot{z}_2\ddot{z}_1\right)}{\left(\dot{z}_1^2 + \dot{z}_2^2\right)^{3/2}}\right). \tag{6}$$

Finally, the steering angular velocity is obtained by differentiating the steering angle:

$$\omega^s = \dot{\varphi}.$$

Let

$$A = \dot{z}_1\ddot{z}_2 - \dot{z}_2\ddot{z}_1, \qquad B = \left(\dot{z}_1^2 + \dot{z}_2^2\right)^{3/2}.$$

Then

$$\varphi = \arctan\left(\frac{lA}{B}\right).$$

The derivative of A is

$$\dot{A} = \dot{z}_1 z_2^{(3)} - \dot{z}_2 z_1^{(3)}.$$

The derivative of B is

$$\dot{B} = 3\left(\dot{z}_1^2 + \dot{z}_2^2\right)^{1/2}\left(\dot{z}_1\ddot{z}_1 + \dot{z}_2\ddot{z}_2\right).$$

Therefore, the steering angular velocity can be written as

$$\omega^s = \frac{l\left(\dot{A}B - A\dot{B}\right)}{B^2 + l^2A^2}.$$

Substituting A, $\dot{A}$, B, and $\dot{B}$ into the above expression gives

$$\omega^s = \frac{l\left[\left(\dot{z}_1 z_2^{(3)} - \dot{z}_2 z_1^{(3)}\right)\left(\dot{z}_1^2 + \dot{z}_2^2\right)^{3/2} - 3\left(\dot{z}_1\ddot{z}_2 - \dot{z}_2\ddot{z}_1\right)\left(\dot{z}_1^2 + \dot{z}_2^2\right)^{1/2}\left(\dot{z}_1\ddot{z}_1 + \dot{z}_2\ddot{z}_2\right)\right]}{\left(\dot{z}_1^2 + \dot{z}_2^2\right)^3 + l^2\left(\dot{z}_1\ddot{z}_2 - \dot{z}_2\ddot{z}_1\right)^2}. \tag{7}$$

Consequently, all the states and inputs of the car-like model can be expressed as functions of the flat output $\mathbf{z}$ and its derivatives:

$$x = z_1, \qquad y = z_2, \qquad \theta = \arctan\left(\frac{\dot{z}_2}{\dot{z}_1}\right), \qquad \varphi = \arctan\left(\frac{lA}{B}\right),$$

and

$$v = \sqrt{\dot{z}_1^2 + \dot{z}_2^2}, \qquad \omega^s = \frac{l\left(\dot{A}B - A\dot{B}\right)}{B^2 + l^2 A^2}.$$

2.2 Flat Output Derivatives in Terms of States and Inputs

The boundary conditions can be written in terms of the vehicle states and inputs. At a given time instant, let

$$\mathbf{x} = \begin{bmatrix} x & y & \theta & \varphi \end{bmatrix}^{\mathrm{T}}, \qquad \mathbf{u} = \begin{bmatrix} v & \omega^s \end{bmatrix}^{\mathrm{T}}.$$

Using the flatness relations, the corresponding flat output and its derivatives are expressed as

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix}. \tag{8}$$

From the kinematic model, the first derivative is

$$\dot{\mathbf{z}} = \begin{bmatrix} \dot{z}_1 \\ \dot{z}_2 \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \end{bmatrix}. \tag{9}$$

The second derivative is obtained by differentiating $\dot{\mathbf{z}}$:

$$\ddot{z}_1 = \dot{v}\cos\theta - v\dot{\theta}\sin\theta,$$

$$\ddot{z}_2 = \dot{v}\sin\theta + v\dot{\theta}\cos\theta.$$

Since

$$\dot{\theta} = \frac{v}{l}\tan\varphi,$$

we obtain

$$\ddot{\mathbf{z}} = \begin{bmatrix} \dot{v}\cos\theta - \dfrac{v^2}{l}\tan\varphi\sin\theta \\ \dot{v}\sin\theta + \dfrac{v^2}{l}\tan\varphi\cos\theta \end{bmatrix}. \tag{10}$$

The third derivative is then obtained by differentiating $\ddot{\mathbf{z}}$. For compactness, define

$$\Omega = \dot{\theta} = \frac{v}{l}\tan\varphi, \qquad \dot{\Omega} = \frac{\dot{v}}{l}\tan\varphi + \frac{v}{l}\sec^2\varphi\,\omega^s, \qquad where\,\sec^2\varphi = \frac{1}{\cos^2\varphi}.$$

Then,

$$\mathbf{z}^{(3)} = \begin{bmatrix} \ddot{v}\cos\theta - 2\dot{v}\Omega\sin\theta - v\dot{\Omega}\sin\theta - v\Omega^2\cos\theta \\ \ddot{v}\sin\theta + 2\dot{v}\Omega\cos\theta + v\dot{\Omega}\cos\theta - v\Omega^2\sin\theta \end{bmatrix}. \tag{11}$$

At the boundary conditions, the longitudinal velocity is assumed to be constant, i.e.,

$$v = \text{const} \quad \Rightarrow \quad \dot{v} = 0, \quad \ddot{v} = 0.$$

Under this assumption, the second derivative of the flat output becomes

$$\ddot{\mathbf{z}} = \begin{bmatrix} -\dfrac{v^2}{l}\tan\varphi\sin\theta \\ \dfrac{v^2}{l}\tan\varphi\cos\theta \end{bmatrix}. \tag{12}$$

Similarly, using
$$\dot{\theta} = \Omega = \frac{v}{l}\tan\varphi, \qquad \dot{\Omega} = \frac{v}{l}\sec^2\varphi\,\omega^s,$$
The third derivative simplifies to
$$\mathbf{z}^{(3)} = \begin{bmatrix} -v\dot{\Omega}\sin\theta - v\Omega^2\cos\theta \\ v\dot{\Omega}\cos\theta - v\Omega^2\sin\theta \end{bmatrix}. \tag{13}$$

3 Trajectory Generating

The trajectory is generated from the flat output
$$\mathbf{z}(t) = \begin{bmatrix} z_1(t) \\ z_2(t) \end{bmatrix} = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}.$$

In this work, the boundary conditions are imposed on the flat output and its derivatives up to the third order at both the initial and final times:
$$\mathbf{z}(0), \quad \dot{\mathbf{z}}(0), \quad \ddot{\mathbf{z}}(0), \quad \mathbf{z}^{(3)}(0),$$
$$\mathbf{z}(T), \quad \dot{\mathbf{z}}(T), \quad \ddot{\mathbf{z}}(T), \quad \mathbf{z}^{(3)}(T).$$

For a Bézier polynomial of degree n, there are $n+1$ control points. Since the boundary conditions are imposed at both the initial and final times, the total number of boundary constraints is $2(r+1)$, where r is the highest derivative order. In this work, the highest derivative order is $r = 3$. Therefore,
$$n + 1 = 2(3 + 1) = 8 => n = 7$$

The Bézier polynomial of degree 7 is defined as
$$\mathbf{z}(s) = \sum_{i=0}^{7} \mathbf{P}_i B_i^7(s), \qquad s \in [0, 1], \tag{14}$$
where
$$B_i^7(s) = \binom{7}{i}(1-s)^{7-i}s^i, \qquad i = 0, 1, \ldots, 7. \tag{15}$$

Since the Bézier parameter is defined over the interval $s \in [0, 1]$, while the physical time evolves over $t \in [0, T]$, where T denotes the total trajectory duration, we introduce the normalized time variable
$$s = \frac{t}{T}.$$

Thus, using the chain rule, the time derivative can be written as
$$\frac{ds}{dt} = \frac{1}{T}.$$

First derivative of the Bézier polynomial
Starting from the Bézier polynomial,
$$\mathbf{z}(s) = \sum_{i=0}^{7} \mathbf{P}_i B_i^7(s),$$
where
$$B_i^7(s) = \binom{7}{i}(1-s)^{7-i}s^i.$$

Differentiating with respect to s, we obtain
$$\frac{d\mathbf{z}}{ds} = \sum_{i=0}^{7} \mathbf{P}_i \frac{d}{ds}\left[\binom{7}{i}(1-s)^{7-i}s^i\right].$$

Using the product rule,

$$\begin{aligned}\frac{dB_i^7(s)}{ds} &= \frac{d}{ds}\left[\binom{7}{i}(1-s)^{7-i}s^i\right] \\ &= \binom{7}{i}\left[-(7-i)(1-s)^{6-i}s^i + i(1-s)^{7-i}s^{i-1}\right].\end{aligned}$$

Expanding the derivative of the Bézier polynomial gives

$$\begin{aligned}\frac{d\mathbf{z}}{ds} &= \sum_{i=0}^{7}\mathbf{P}_i\frac{dB_i^7(s)}{ds} \\ &= \sum_{i=0}^{7}\mathbf{P}_i\binom{7}{i}\left[-(7-i)(1-s)^{6-i}s^i + i(1-s)^{7-i}s^{i-1}\right] \\ &= \sum_{i=0}^{7}\left[-\mathbf{P}_i\binom{7}{i}(7-i)(1-s)^{6-i}s^i + \mathbf{P}_i\binom{7}{i}i(1-s)^{7-i}s^{i-1}\right].\end{aligned}$$

Using

$$\binom{7}{i}(7-i) = 7\binom{6}{i}, \qquad \binom{7}{i}i = 7\binom{6}{i-1},$$

we obtain

$$\begin{aligned}\frac{d\mathbf{z}}{ds} &= \sum_{i=0}^{7}\left[-7\mathbf{P}_i\binom{6}{i}(1-s)^{6-i}s^i + 7\mathbf{P}_i\binom{6}{i-1}(1-s)^{7-i}s^{i-1}\right] \\ &= 7\sum_{i=0}^{7}\mathbf{P}_i\left[B_{i-1}^6(s) - B_i^6(s)\right].\end{aligned}$$

Expanding the summation gives

$$\begin{aligned}\frac{d\mathbf{z}}{ds} = 7\big[&\mathbf{P}_0(B_{-1}^6 - B_0^6) + \mathbf{P}_1(B_0^6 - B_1^6) + \cdots \\ &+ \mathbf{P}_6(B_5^6 - B_6^6) + \mathbf{P}_7(B_6^6 - B_7^6)\big].\end{aligned}$$

Since $B_{-1}^6(s) = 0$ and $B_7^6(s) = 0$, we have

$$\begin{aligned}\frac{d\mathbf{z}}{ds} = 7\big[&(\mathbf{P}_1 - \mathbf{P}_0)B_0^6(s) + (\mathbf{P}_2 - \mathbf{P}_1)B_1^6(s) + \cdots \\ &+ (\mathbf{P}_7 - \mathbf{P}_6)B_6^6(s)\big].\end{aligned}$$

Thus, the first derivative with respect to s is

$$\frac{d\mathbf{z}}{ds} = 7\sum_{i=0}^{6}\left(\mathbf{P}_{i+1} - \mathbf{P}_i\right)B_i^6(s).$$

Since $s = t/T$, the first time derivative is

$$\dot{\mathbf{z}}(t) = \frac{7}{T}\sum_{i=0}^{6}\left(\mathbf{P}_{i+1} - \mathbf{P}_i\right)B_i^6(s). \tag{16}$$

The second time derivative is

$$\ddot{\mathbf{z}}(t) = \frac{42}{T^2}\sum_{i=0}^{5}\left(\mathbf{P}_{i+2} - 2\mathbf{P}_{i+1} + \mathbf{P}_i\right)B_i^5(s). \tag{17}$$

The third time derivative is

$$\mathbf{z}^{(3)}(t) = \frac{210}{T^3}\sum_{i=0}^{4}\left(\mathbf{P}_{i+3} - 3\mathbf{P}_{i+2} + 3\mathbf{P}_{i+1} - \mathbf{P}_i\right)B_i^4(s). \tag{18}$$

At $s = 0$, the first four control points are obtained from the initial boundary conditions:

$$\mathbf{P}_0 = \mathbf{z}(0), \tag{19}$$

$$\mathbf{P}_1 = \mathbf{z}(0) + \frac{T}{7}\dot{\mathbf{z}}(0), \tag{20}$$

$$\mathbf{P}_2 = \mathbf{z}(0) + \frac{2T}{7}\dot{\mathbf{z}}(0) + \frac{T^2}{42}\ddot{\mathbf{z}}(0), \tag{21}$$

$$\mathbf{P}_3 = \mathbf{z}(0) + \frac{3T}{7}\dot{\mathbf{z}}(0) + \frac{T^2}{14}\ddot{\mathbf{z}}(0) + \frac{T^3}{210}\mathbf{z}^{(3)}(0). \tag{22}$$

At $s = 1$, the remaining control points are obtained from the final boundary conditions:

$$\mathbf{P}_7 = \mathbf{z}(T), \tag{23}$$

$$\mathbf{P}_6 = \mathbf{z}(T) - \frac{T}{7}\dot{\mathbf{z}}(T), \tag{24}$$

$$\mathbf{P}_5 = \mathbf{z}(T) - \frac{2T}{7}\dot{\mathbf{z}}(T) + \frac{T^2}{42}\ddot{\mathbf{z}}(T), \tag{25}$$

$$\mathbf{P}_4 = \mathbf{z}(T) - \frac{3T}{7}\dot{\mathbf{z}}(T) + \frac{T^2}{14}\ddot{\mathbf{z}}(T) - \frac{T^3}{210}\mathbf{z}^{(3)}(T). \tag{26}$$

4 Result

4.1 Automatic Computation of Boundary Conditions from Waypoint Geometry

In practice, the user only specifies the Cartesian position and desired longitudinal velocity at each waypoint:

$$\mathbf{w}_i = \begin{bmatrix} x_i & y_i & v_i \end{bmatrix}^\top .$$

The remaining variables required by the flatness-based formulation, namely the heading angle θ_i, the steering angle φ_i, and the steering angular velocity ω_i^s, are automatically computed from the waypoint geometry.

The local tangent direction at each waypoint is estimated from the local geometric direction of the path. Let

$$\mathbf{p}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

denote the Cartesian position of waypoint i.

For the first waypoint, the tangent direction is taken from the first path segment:

$$\tilde{\mathbf{h}}_0 = \mathbf{p}_1 - \mathbf{p}_0.$$

The corresponding unit heading direction is then

$$\hat{\mathbf{h}}_0 = \frac{\tilde{\mathbf{h}}_0}{\left\|\tilde{\mathbf{h}}_0\right\|}.$$

For the last waypoint, the tangent direction is taken from the last path segment:

$$\tilde{\mathbf{h}}_{N-1} = \mathbf{p}_{N-1} - \mathbf{p}_{N-2}.$$

The corresponding unit heading direction is then

$$\hat{\mathbf{h}}_{N-1} = \frac{\tilde{\mathbf{h}}_{N-1}}{\left\|\tilde{\mathbf{h}}_{N-1}\right\|}.$$

For an intermediate waypoint, two local unit direction vectors are constructed:

$$\mathbf{d}_i^- = \frac{\mathbf{p}_i - \mathbf{p}_{i-1}}{\|\mathbf{p}_i - \mathbf{p}_{i-1}\|}, \qquad \mathbf{d}_i^+ = \frac{\mathbf{p}_{i+1} - \mathbf{p}_i}{\|\mathbf{p}_{i+1} - \mathbf{p}_i\|}.$$

Here, $\mathbf{d}_i^-$ represents the normalized incoming direction, whereas $\mathbf{d}_i^+$ represents the normalized outgoing direction. The non-normalized local tangent direction is approximated by

$$\tilde{\mathbf{h}}_i = \mathbf{d}_i^- + \mathbf{d}_i^+.$$

The corresponding unit heading direction is then obtained by normalization:

$$\hat{\mathbf{h}}_i = \frac{\tilde{\mathbf{h}}_i}{\left\|\tilde{\mathbf{h}}_i\right\|}.$$

This construction can be interpreted as a smoothed line-of-sight direction: the heading does not only point toward the next waypoint, but also takes into account the direction from which the vehicle arrives at the current waypoint.

By construction, the unit heading direction satisfies

$$\hat{\mathbf{h}}_i = \begin{bmatrix} \cos\theta_i \\ \sin\theta_i \end{bmatrix}.$$

Finally, the heading angle is extracted from the unit heading direction using the two-argument arctangent:

$$\theta_i = \operatorname{atan2}\left(\hat{h}_{i,y}, \hat{h}_{i,x}\right). \tag{27}$$

The function atan2 is used instead of $\arctan(\hat{h}_{i,y}/\hat{h}_{i,x})$ because it correctly determines the quadrant of the heading angle and avoids division by zero when $\hat{h}_{i,x} = 0$.

Therefore, the heading angle is not manually specified by the user, but is automatically inferred from the local geometry of the waypoint sequence.

Although the path curvature can be defined as

$$\kappa_i = \frac{\dot{\theta}_i}{v_i},$$

This expression cannot be directly used at this stage. Indeed, before the trajectory is generated, only the waypoint positions and velocities (x_i, y_i, v_i) are known. The continuous functions $\theta(t)$ and $\dot{\theta}(t)$ are not yet available. Therefore, the curvature is first estimated from the geometry of three consecutive waypoints. This geometric approximation provides the local bending of the path before the full trajectory is constructed.

In the waypoint-based formulation, the curvature at an intermediate waypoint is approximated from three consecutive points:

$$\mathbf{p}_{i-1}, \qquad \mathbf{p}_i, \qquad \mathbf{p}_{i+1}.$$

These three points define two consecutive path segments. The first segment is

$$\mathbf{a}_i = \mathbf{p}_i - \mathbf{p}_{i-1},$$

and the second segment is

$$\mathbf{b}_i = \mathbf{p}_{i+1} - \mathbf{p}_i.$$

The signed determinant of two planar vectors is defined as

$$\det(\mathbf{a}_i, \mathbf{b}_i) = a_{i,x}b_{i,y} - a_{i,y}b_{i,x}.$$

The curvature is then approximated as

$$\kappa_i = \frac{2\det(\mathbf{a}_i, \mathbf{b}_i)}{\|\mathbf{a}_i\|\,\|\mathbf{b}_i\|\,\|\mathbf{p}_{i+1} - \mathbf{p}_{i-1}\|}. \tag{28}$$

Equivalently,

$$\kappa_i = \frac{2\det(\mathbf{p}_i - \mathbf{p}_{i-1}, \mathbf{p}_{i+1} - \mathbf{p}_i)}{\|\mathbf{p}_i - \mathbf{p}_{i-1}\| \, \|\mathbf{p}_{i+1} - \mathbf{p}_i\| \, \|\mathbf{p}_{i+1} - \mathbf{p}_{i-1}\|}. \tag{29}$$

This formula corresponds to the signed curvature of the circle passing through the three consecutive waypoints. It can be obtained from the circumcircle relation

$$R = \frac{abc}{4S},$$

where R is the radius of the circle, a, b, and c are the side lengths of the triangle formed by the three waypoints, and S is the area of this triangle. Since curvature is the inverse of the radius,

$$\kappa = \frac{1}{R} = \frac{4S}{abc}.$$

For the three waypoints considered here, the side lengths are

$$a = \|\mathbf{p}_i - \mathbf{p}_{i-1}\|, \qquad b = \|\mathbf{p}_{i+1} - \mathbf{p}_i\|, \qquad c = \|\mathbf{p}_{i+1} - \mathbf{p}_{i-1}\|.$$

The signed area of the triangle is

$$S_{\text{signed}} = \frac{1}{2}\det(\mathbf{a}_i, \mathbf{b}_i).$$

Therefore, the signed curvature becomes

$$\kappa_i = \frac{4S_{\text{signed}}}{abc} = \frac{2\det(\mathbf{a}_i, \mathbf{b}_i)}{\|\mathbf{a}_i\| \, \|\mathbf{b}_i\| \, \|\mathbf{p}_{i+1} - \mathbf{p}_{i-1}\|}.$$

The determinant $\det(\mathbf{a}_i, \mathbf{b}_i)$ represents the signed area of the parallelogram formed by the two consecutive segments. Equivalently, it is twice the signed area of the triangle formed by the three waypoints. Therefore, its sign indicates the turning direction. A positive determinant corresponds to a left turn, while a negative determinant corresponds to a right turn. If the three points are aligned, the determinant is zero and the curvature is also zero.

At the first and last waypoints, the curvature is set to zero:

$$\kappa_0 = 0, \qquad \kappa_{N-1} = 0.$$

This choice imposes zero steering angle at the beginning and at the end of the trajectory, which avoids an initial or final sharp steering command.

Using the geometric relation between curvature and steering angle, the steering angle at each waypoint is computed as

$$\varphi_i = \arctan(l\kappa_i). \tag{30}$$

This relation comes from the car-like kinematic model:

$$\dot{\theta} = \frac{v}{l}\tan\varphi.$$

Since the curvature of the path is

$$\kappa = \frac{\dot{\theta}}{v},$$

we obtain

$$\kappa = \frac{\tan\varphi}{l}.$$

Hence,

$$\tan\varphi = l\kappa,$$

and therefore

$$\varphi = \arctan(l\kappa).$$

The duration of each trajectory segment is then computed from the distance between two consecutive waypoints and the desired speeds at these waypoints. The distance of segment i is

$$d_i = \|\mathbf{p}_{i+1} - \mathbf{p}_i\|.$$

The average speed over this segment is approximated by

$$\bar{v}_i = \frac{|v_i| + |v_{i+1}|}{2}.$$

Therefore, the travelling time required by the speed condition is

$$T_{v,i} = \frac{d_i}{\bar{v}_i}.$$

However, the segment duration must also be sufficiently large to respect the steering angular velocity limit. If the steering angle changes from φ_i to φ_{i+1}, the minimum time required by this steering variation is

$$T_{\omega,i} = \frac{|\varphi_{i+1} - \varphi_i|}{\omega^s_{\max}}.$$

The final duration of segment i is selected as

$$T_i = \alpha \max\left(T_{v,i}, T_{\omega,i}, T_{\min}\right), \tag{31}$$

where α is a safety factor and $T_{\min}$ is the minimum allowed segment duration.

After the segment durations are known, the steering angular velocity at each waypoint is approximated from the variation of the steering angle. For the first waypoint,

$$\omega^s_0 = \frac{\varphi_1 - \varphi_0}{T_0}.$$

For the last waypoint,

$$\omega^s_{N-1} = \frac{\varphi_{N-1} - \varphi_{N-2}}{T_{N-2}}.$$

For an intermediate waypoint, a centered approximation is used:

$$\omega^s_i = \frac{\varphi_{i+1} - \varphi_{i-1}}{T_{i-1} + T_i}. \tag{32}$$

Thus, each user-defined waypoint

$$\begin{bmatrix} x_i & y_i & v_i \end{bmatrix}^\top$$

is converted into a complete boundary waypoint:

$$\begin{bmatrix} x_i & y_i & \theta_i & \varphi_i & v_i & \omega^s_i \end{bmatrix}^\top.$$

Finally, the generated trajectory must satisfy the physical steering limits of the QCar2:

$$|\varphi(t)| \leq \varphi_{\max}, \qquad |\omega^s(t)| \leq \omega^s_{\max}. \tag{33}$$

In this implementation, the limits are

$$\varphi_{\max} = 0.5236 \text{ rad}, \qquad \omega^s_{\max} = 2.0 \text{ rad/s}.$$

If these constraints are violated, it means that the waypoint geometry requires a turn that is too sharp, or that the steering angle changes too quickly. Therefore, the waypoint geometry must be smoothed, or the segment duration must be increased.

4.2 Simulation Results

In this section, the proposed flatness-based trajectory generation method is validated through numerical simulation. A set of four waypoints is provided as input in the form

$$\begin{bmatrix} x_i & y_i & v_i \end{bmatrix}^\top.$$

In this simulation, the selected waypoints are

$$\begin{aligned}
\mathbf{w}_0 &= \begin{bmatrix}0.0 & 0.0 & 0.35\end{bmatrix}^\top, \\
\mathbf{w}_1 &= \begin{bmatrix}-1.0 & 0.2 & 0.35\end{bmatrix}^\top, \\
\mathbf{w}_2 &= \begin{bmatrix}-2.0 & 0.2 & 0.30\end{bmatrix}^\top, \\
\mathbf{w}_3 &= \begin{bmatrix}-3.5 & -0.3 & 0.05\end{bmatrix}^\top.
\end{aligned}$$

and the remaining states and inputs are automatically computed as described in the previous subsection.

The resulting trajectory and corresponding state and input profiles are shown in Figures 2,3, 4, 5, and 6.

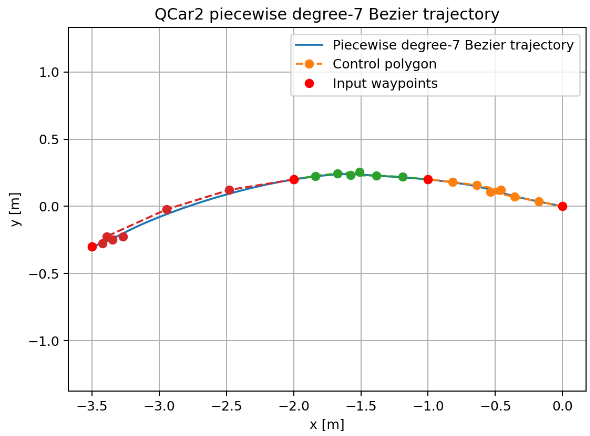

Figure 2: Generated piecewise degree-7 Bézier trajectory.

Figure 2 shows the generated trajectory in the Cartesian plane. The curve smoothly interpolates the four input waypoints while ensuring continuity up to the third derivative, as required by the flatness-based formulation.

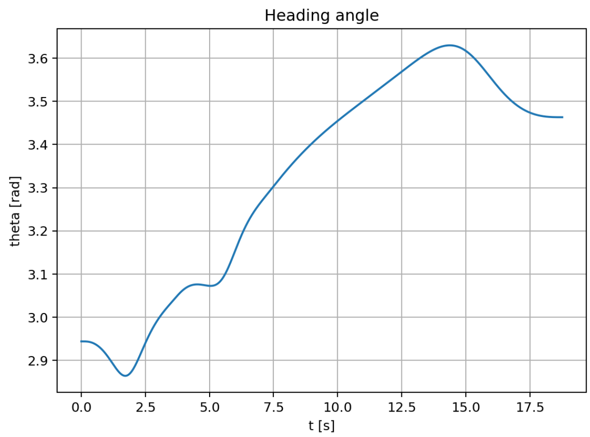

Figure 3: Heading angle (θ)

The heading angle varies smoothly along the trajectory without discontinuities, which confirms that the tangent direction computed from the waypoint geometry is consistent.

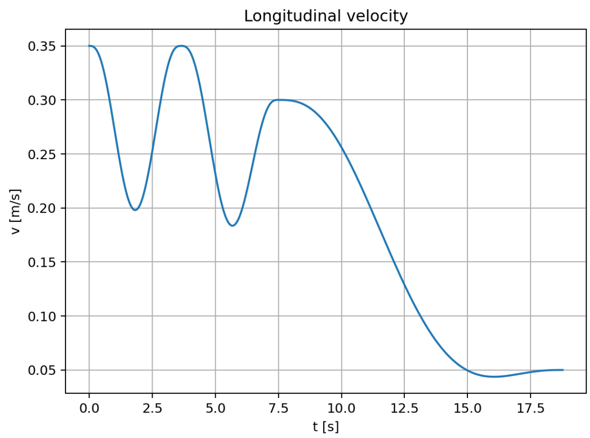

Figure 4: Longitudinal velocity

The longitudinal velocity follows the values specified at the waypoints while remaining smooth along the trajectory due to the Bézier parametrization.

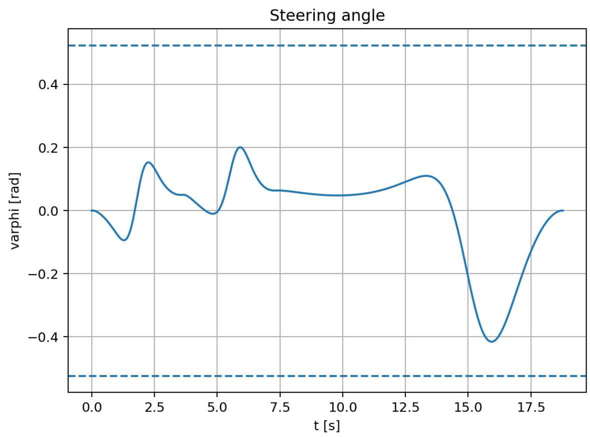

Figure 5: Steering angle (φ) with bounds.

The steering angle remains within the physical limits imposed by the QCar2 model, i.e.,

$$|\varphi(t)| \leq 0.5236 \text{ rad}.$$

This confirms that the curvature induced by the waypoint geometry is feasible.

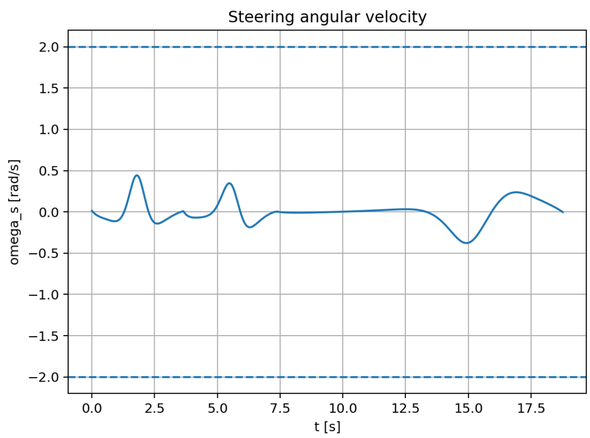

Figure 6: Steering angular velocity (ω^s) profile with bounds.

Similarly, the steering angular velocity satisfies the constraint

$$|\omega^s(t)| \leq 2.0 \text{ rad/s},$$

which demonstrates that the automatically computed segment durations are sufficient to ensure actuator feasibility.

If the last waypoint is changed too aggressively, for example

$$\begin{aligned}
\mathbf{w}_0 &= \begin{bmatrix} 0.0 & 0.0 & 0.35 \end{bmatrix}^\top, \\
\mathbf{w}_1 &= \begin{bmatrix} -1.0 & 0.2 & 0.35 \end{bmatrix}^\top, \\
\mathbf{w}_2 &= \begin{bmatrix} -2.0 & 0.2 & 0.30 \end{bmatrix}^\top, \\
\mathbf{w}_3 &= \begin{bmatrix} -3.5 & -1.0 & 0.05 \end{bmatrix}^\top,
\end{aligned}$$

The generated trajectory requires a sharper turn near the final segment. As a result, the steering angle exceeds the physical limit:

$$|\varphi(t)| > \varphi_{\max}.$$

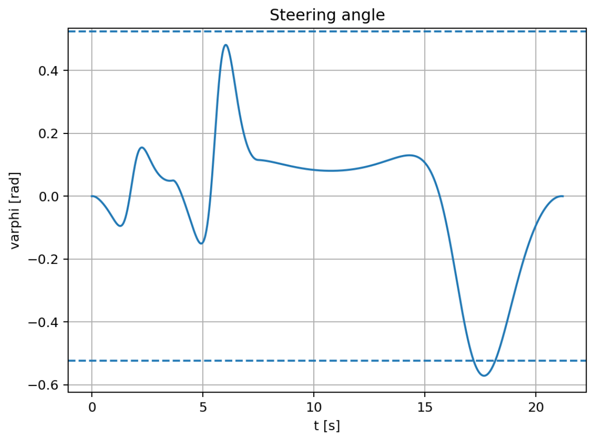

Figure 7: Steering angle violation when the last waypoint is too aggressive.

This result shows that the waypoint geometry directly affects the curvature of the generated path. Therefore, if the waypoint position creates a turn that is too sharp, the corresponding steering angle may violate the QCar2 steering limit. In this case, the waypoint should be smoothed or moved closer to the previous path direction.

The proposed trajectory generation method is further validated on ROS 2. The simulation is conducted on a previously scanned map of room D215, which is used as the navigation reference.

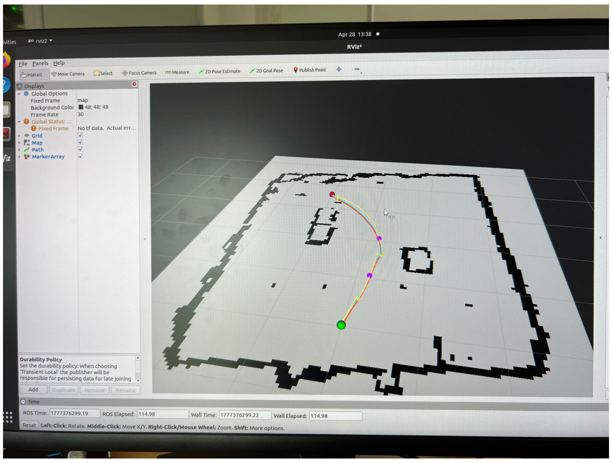

Figure 8: Trajectory generated and visualized in ROS2 on the scanned map of room D215

As shown in Fig. 8, the generated trajectory follows the desired path smoothly while avoiding obstacles present in the environment. This result demonstrates the applicability of the flatness-based Bézier trajectory generation approach in a realistic setting using ROS2.

Overall, the results show that the proposed method can generate a smooth, dynamically feasible trajectory from a minimal set of user-defined inputs (x, y, v). The automatic computation of the remaining variables ensures consistency with the vehicle kinematics while respecting the physical constraints of the system.